

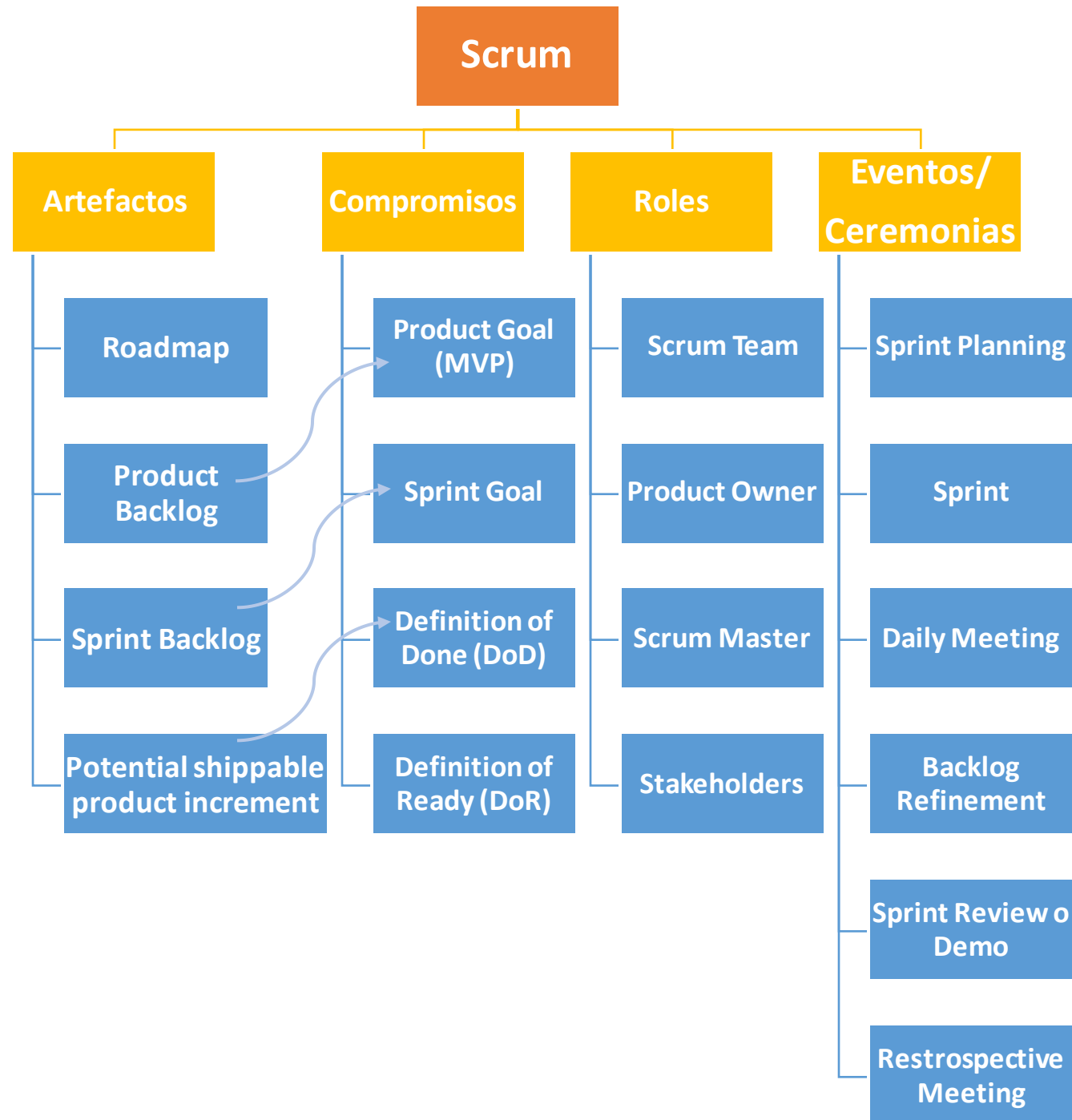
PRACTICA: temas a ver y aplicar

- ✱ Armado, planificación, estimación, priorización y dependencias de User Stories y backlogs
- ✱ INVEST
- ✱ Técnicas y Patrones de división de Historias de Usuario (Slicing)
- ✱ Criterios de aceptación
- ✱ SMART y Gherkin
- ✱ DoD y DoR
- ✱ Refinamiento del Producto Backlog
- ✱ Mapas de US

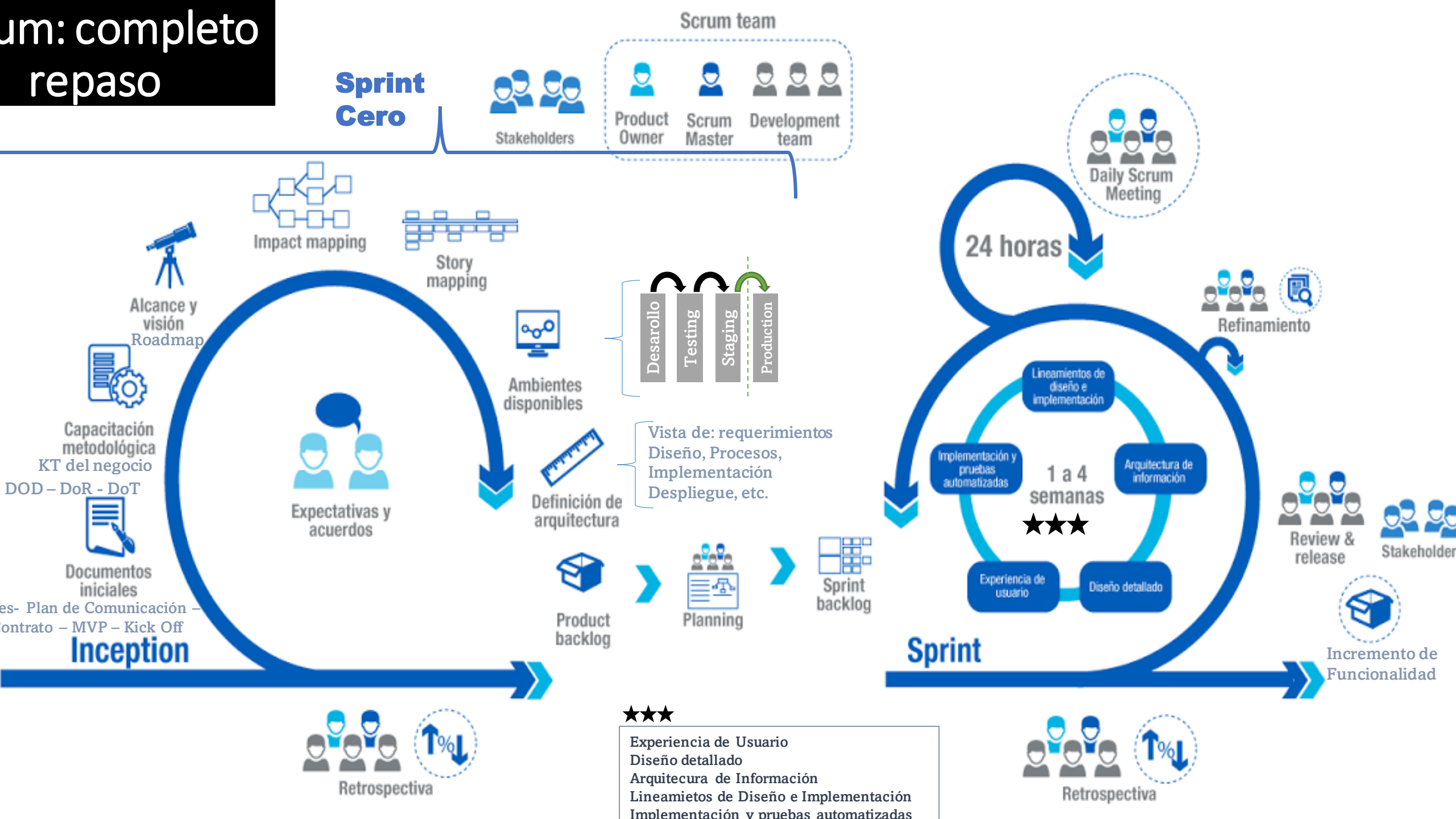
repaso



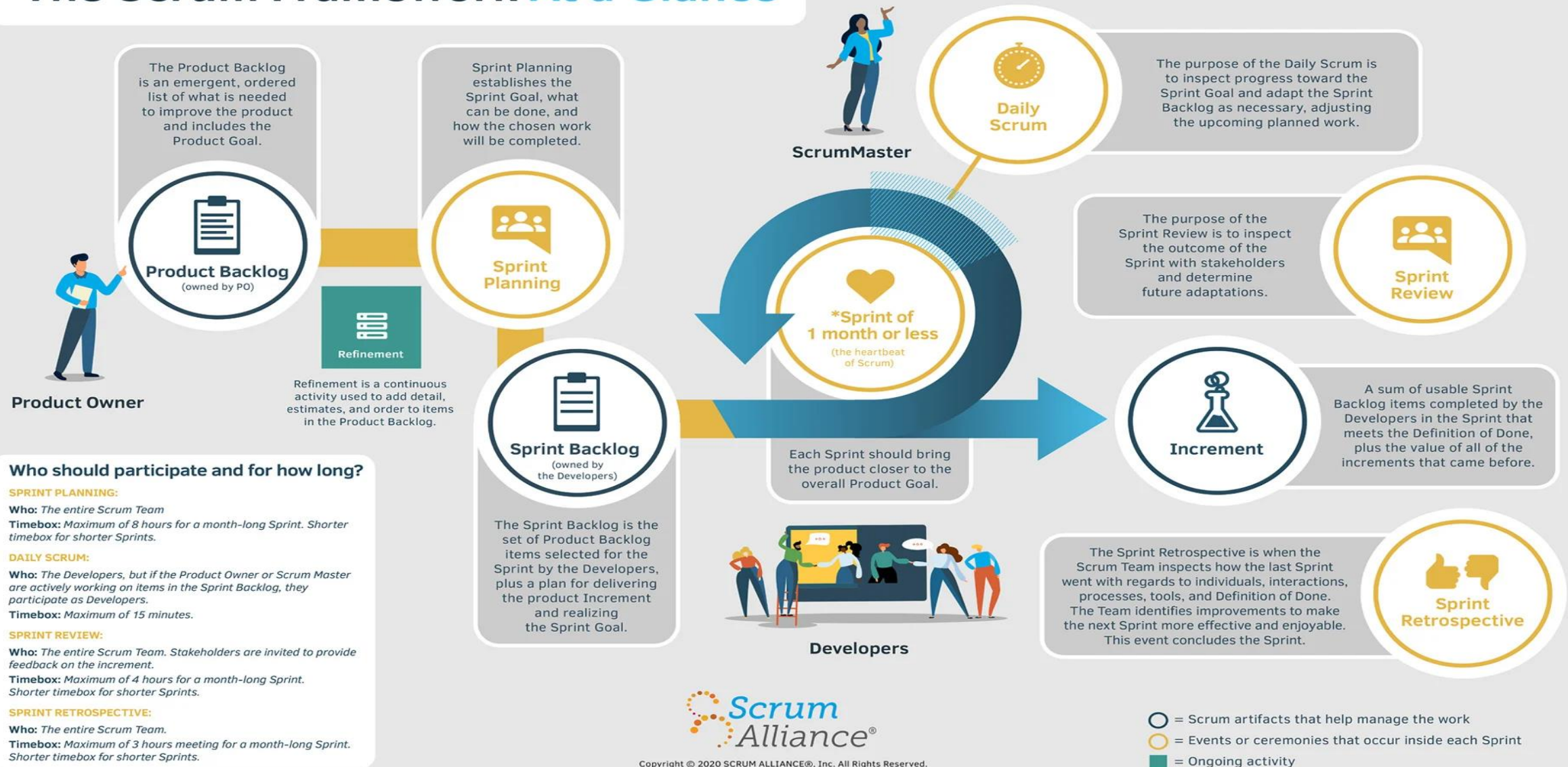
MVP se refiere al producto mínimo viable (en inglés, "minimum viable product"), o sea, un producto con las características esenciales para satisfacer a los clientes.



um: completo repass

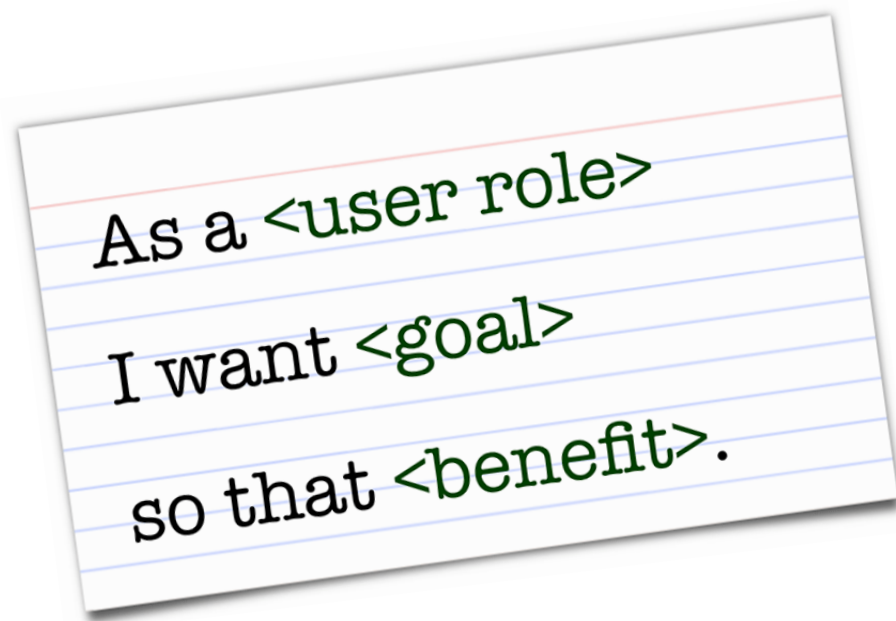


The Scrum Framework At a Glance



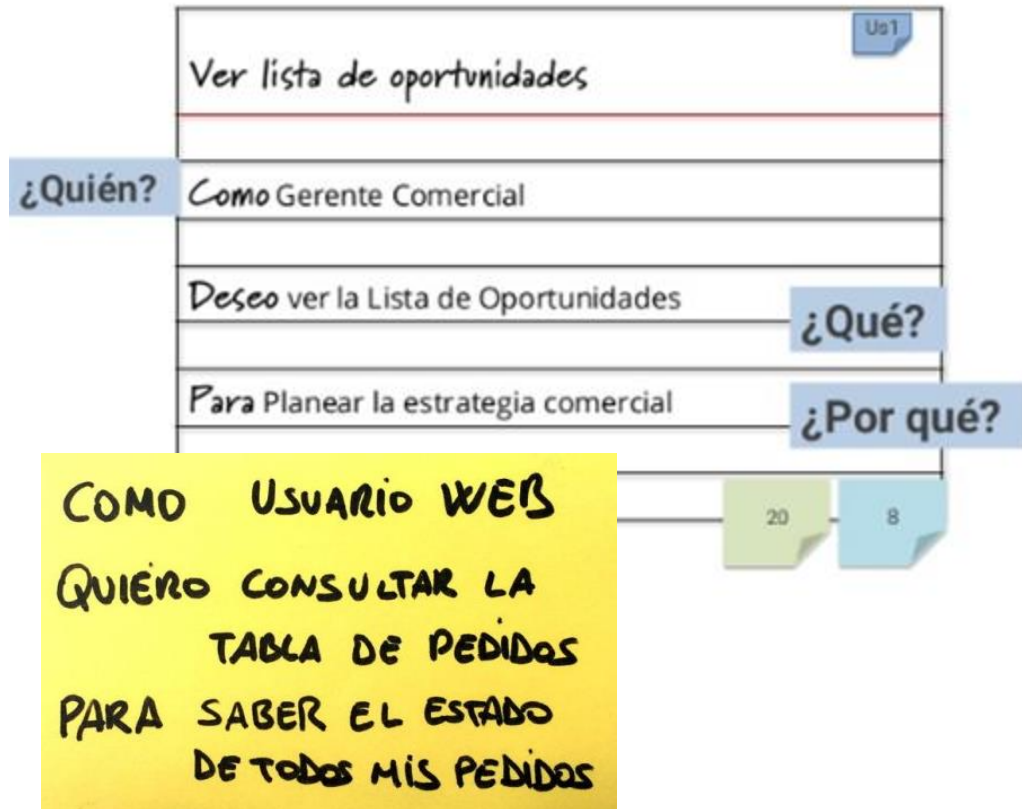
Historia de Usuario o User Story

- Las historias representan un concepto verificable y se define como una "**promesa de una conversación**" o una "**descripción de una característica**", representando una "**funcionalidad de aplicación**" para un usuario y que brinda un beneficio (ROL + FUNCIONALIDAD + BENEFICIO).
- Se escriben siempre con lenguaje de negocio y respetando la siguiente pauta de estructura:
 - **COMO** <ROL>
 - **QUIERO** <FUNCIONALIDAD/OBJETIVO>
 - **PARA QUE** <BENEFICIO/VALOR DE NEGOCIO>
- Por ejemplo:
 - **COMO** médico pediatra de la clínica ABC
 - **QUIERO** consultar las historias clínicas
 - **PARA PODER** dar un diagnóstico más acertado de mis pacientes
 - **COMO** responsable del área de Compras
 - **QUIERO** organizar mi trabajo
 - **PARA PODER** tener mayor control de los proveedores



As a <user role>
I want <goal>
so that <benefit>.

Ejemplos



Como usuario final, quiero poder restablecer mi contraseña, de modo que pueda tener acceso a mi cuenta de nuevo.

Estimación: 5 SP

Como un cliente,
quiero consultar el catálogo
Para Poder
encontrar el producto que
deseo comprar.

Historia: Responder a comentarios

Como: Lector del Blog

Quiero: responder a comentarios de otros lectores

Para: mantenerme en contacto con los demás usuarios del blog

Historia: Agregar comentarios

Como: Lector del Blog

Quiero: adicionar comentarios a las entradas

Para: mantenerme en contacto con el autor del blog

Historia: Recibir alertas

Como: Lector del Blog

Quiero: recibir alertas cuando otros hagan comentarios a las entradas de mi elección

Para: enterarme de lo que otros piensan sobre los temas de interés

Artefactos que se usan y generan: Product Backlog

Lista de todos los requerimientos o necesidades para el producto (historia de usuario), presentados de manera priorizada.

- A cargo del Product Owner.
- Nunca esta completo, solo hasta que el producto salga de funcionamiento.
- Esta constantemente cambiando, adaptándose a las necesidades del cliente
- Se pueden encontrar:
 - Características del Producto
 - Funcionalidades
 - Requerimientos
 - Mejoras
 - Correcciones



Historia de Usuario o User Story

¿QUIÉN SE BENEFICIA?

¿QUÉ SE QUIERE?

¿CUÁL ES EL BENEFICIO?

US: es una descripción concisa y escrita de una funcionalidad que será valiosa para el usuario (o propietario) del software.

Como **ROL** quiero **ALGO** para poder **BENEFICIO**

As a _____ I want to _____ So that _____

Persona Action Expected outcome

As a **registered user** I want to **log in**
so I can **access subscriber-only content**

Historia de Usuario: por qué?

Historias de usuario

¿Por
qué?

Utilizamos las historias de usuario porque siguen los principios básicos de requerimientos ágiles:

Potencian la participación del equipo en la toma de decisiones.

Se crean y evolucionan a medida que el proyecto avanza.

Son peticiones concretas y pequeñas.

Contiene la información imprescindible. Menos es más.

Apoyan la cooperación, colaboración y conversación entre los miembros del equipo, lo que es fundamental.

Historia de Usuario: por qué?



Historia de Usuario o User Story: MODELO INVEST

Las historias de usuario **son una forma de definir un requisito** con los objetivos:

Independiente: Debe ser totalmente atómica en su definición

Negociable: Ambiguas en su enunciado para poder ser debatidas. La concreción vendrá dada por los criterios de validación.

Valorable: Es importante conocer el valor que aportan al proyecto, poder valorar la cantidad de trabajo que suponen para poder planificar y gestionar el proyecto.

Estimable: debe de poder ser estimada con la precisión suficiente para ayudar al cliente, usuario o propietario del producto a priorizar y planificar su implementación. La estimación generalmente la realizará el equipo de trabajo y está directamente relacionada con el tamaño de la historia de usuario (una historia de usuario de gran tamaño es más difícil de estimar) y con el conocimiento del equipo de la necesidad expresada (en el caso de falta de conocimiento, serán necesarias mas fases de conversación acerca de la misma).

Small o pequeña: Para poder hacernos una idea rápida una historia de usuario debería poderse llevar a cabo durante un tiempo superior a dos días e inferior a una semana.

Testeable Es obligatorio verificar que el software cumple con la funcionalidad acordada, se hace mediante los criterios de validación. La historia de usuario debería ser capaz de ser probada (fase confirmación de la historia de usuario). Si el cliente o usuario no sabe como probar la historia de usuario significa que no es del todo clara o que no es valiosa. Si el equipo no puede probar una historia de usuario nunca sabrá si la ha terminado o no.

Cómo se escriben la US?

Definir quién utilizara la funcionalidad a desarrollar

- Es útil imaginarnos qué características tienen las personas que usarán el producto, y detallar su necesidad y problemas actuales para dar lugar al entendimiento sobre sus expectativas reales.
- Para poder identificar más fácilmente quién es nuestro usuario podemos hacernos preguntas como ¿Para qué usuario estamos trabajando en esta historia? ¿Qué hace, en qué trabaja, cómo y dónde vive, cuántos años tiene? ¿Qué tecnología sabe usar? ¿Le sería fácil aprender? ¿Para qué quiere usar la aplicación? ¿Qué problemas se le presentan actualmente para resolver su necesidad?

Especificar qué producto quiere el usuario

- Las historias de usuario deben describir qué se espera como salida de la implementación, y **cómo se ve beneficiado el usuario final**. Se expresa en lenguaje natural y sencillo, para poder conversar directamente con ellos sobre el tema. Hay que considerar que los usuarios finales tienden a desconocen el lenguaje técnico, por lo que deben evitarse las palabras difíciles. Al usuario no le importa qué tecnología se usa, lo que busca es que le ayudemos a resolver su problema, y cómo usará esta solución; desea que sea simple, intuitiva y fácil de usar.

Cómo se escriben la US?

Para qué utilizará el producto

- En este sentido es importante definir el contexto donde surge la historia que se está creando. Esto ayudará a entender el valor agregado que se dará, establecer el objetivo de construcción, y aporta la posibilidad de explorar otras alternativas para llegar al mismo fin.

Los criterios de aceptación

- Aquí se especifica qué salidas obtendremos cuando finalice el proceso de ejecución de la funcionalidad, y nos sirve para verificar que está terminada la funcionalidad. Está relacionada con las pruebas que se realizarán para verificar el cumplimiento de la expectativa de diseño, usabilidad, rendimiento, y la satisfacción del usuario.

Y finalmente, los comentarios

- Las historias de usuarios facilitan la interacción permanente con el cliente para verificar que lo que estamos construyendo está de acuerdo a sus expectativas. Esta negociación se va registrando según se necesite como comentarios o notas adicionales a tener en cuenta. Esta forma de trabajo permite colaborar y mantener una comunicación fluida entre los miembros del equipo. Así, la historia va cambiando durante la marcha de la iteración y se va perfeccionando en conjunto.

Priorización de las US

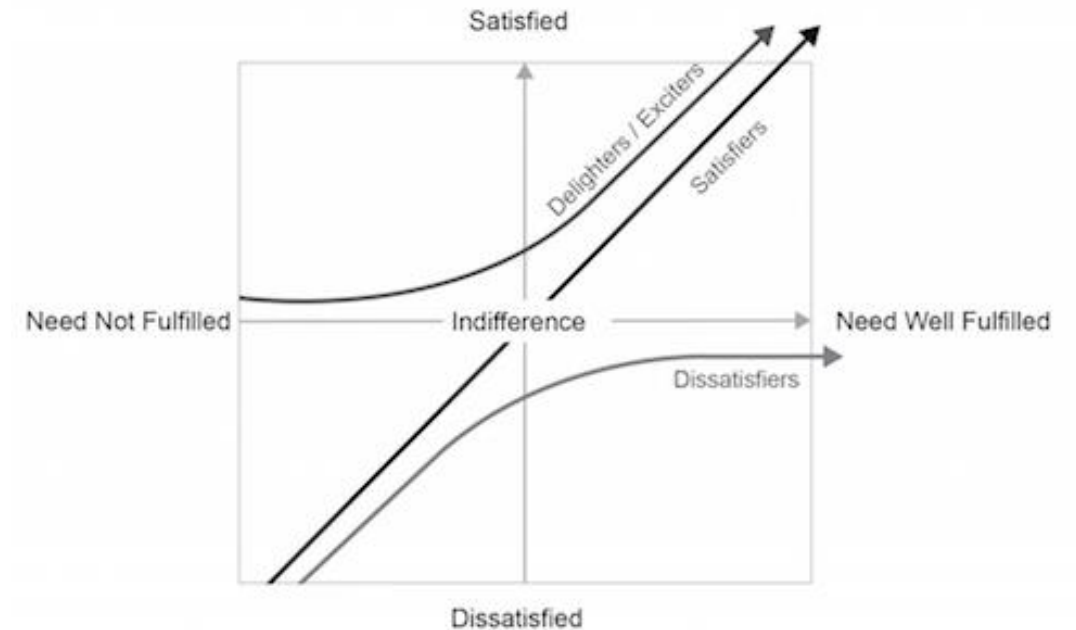
- Algunas Técnicas utilizadas para **priorizar las Historias de Usuarios** o requisitos en el Backlog de Producto Priorizado, según la base del Valor para el negocio:

1.- Esquema de Priorización MoSCoW

Priorización de las US

- Algunas Técnicas utilizadas para priorizar las Historias de Usuarios o requisitos en el Backlog de Producto Priorizado, según la base del Valor para el negocio: -cont.

4.- Análisis Kano



Análisis Kano

Dependencias entre US

Existen numerosos tipos de dependencias: obligatorias y discrecionales, internas y externas, o alguna combinación de estas dependencias. Por ejemplo: una dependencia puede ser ambas, obligatoria y externa.

Conclusiones

- En proyectos grandes, la correcta definición de las dependencias ayuda a los Equipos Scrum a determinar cuál de sus decisiones puede impactar a otros equipos. También puede influir sobre el orden relativo en el cual un solo Equipo Scrum ejecuta sus respectivas tareas para crear los Entregables del Sprint.

User Story: Jerarquía en el Backlog o desgranamiento

Roadmap

Theme 1

Initiatives

Epics

Stories

Theme 2

Initiatives

Epics

Stories

Theme 3

Initiatives

Epics

Stories

Nivel de
planifica-
ción

1/4 to 1 year

Long-term Planning (Roadmap & Release Plan)

1 to 4 week

Sprint Planning

Day

Daily

time

EPIC o Epica: iniciativa global que incluye múltiples historias y que podría abarcar muchos Sprints.

THEME

EPIC

EPIC

USER
STORIES

USER
STORIES

USER
STORIES

TASK

TASK

TASK

TASK

TASK

TASK

TASK

As a "power user", I can
"have my reports" on my
"dashboard"

As a "user", I can "backup"
my "hard drive."

Técnicas y Patrones de división de Historias de Usuario (Slicing)

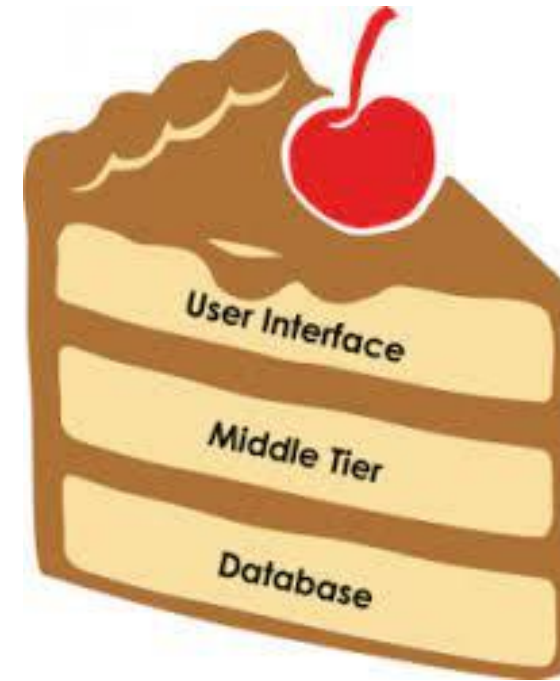
- Existen varias definiciones de como podemos agrupar el alcance por medio de ítems de trabajo que pueden tomar varios tipos, la mas popular es la siguiente:



- Para lograr descomponer los ítems se aplican técnicas **de Slicing que permiten descomponer el trabajo en ítems mas pequeños.**

Técnicas y Patrones de división de Historias de Usuario (Slicing)

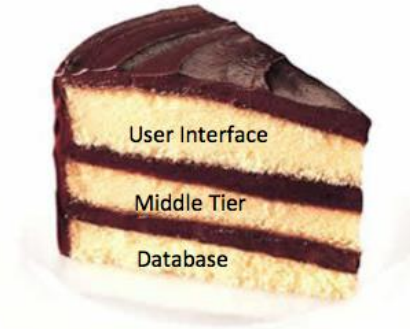
- La incertidumbre de los ítems de trabajo es proporcional a su tamaño, a **mayor tamaño mayor incertidumbre**, ya que agrupa demasiadas definiciones que entenderlas, detallarlas, estimarlas y planificarlas, se vuelve una tarea difícil, por ello se recomienda descomponer el trabajo en ítems mas pequeños que ayuden al negocio y al equipo a tener una mejor priorización y estimación al momento de planificar.
- Cuando aplicamos Slicing tenemos las siguientes **ventajas**:
 - Entrega de valor con más frecuencia.
 - Planificación más fácil.
 - Feedback frecuente del negocio.
 - Mejor sincronización entre personas y equipos.
 - Mejores prioridades.
 - Menos riesgo.
 - Integraciones menos complicadas.
 - Entendimiento compartido
- Si su “corte” (división) da como resultado múltiples historias más pequeñas, cada una de las cuales se deben entregar para completar la más grande, en realidad no está “cortando” (dividiendo), se está descomponiendo.



Patrones de División de Historias de Usuario

Desglosado por:

1. Pasos del flujo de trabajo
2. Reglas de negocio, ya sean comerciales implícitas o explícitas
3. Flujo o camino feliz/infeliz
4. Opciones de entrada/plataforma
5. Tipos de datos o parámetros que devuelven o los parámetros que deben manejar
6. Operaciones
 - Los PBI a menudo involucran una serie de operaciones predeterminadas, como Crear, Leer, Actualizar o Borrado (comúnmente abreviadas como CRUD).
7. Escenarios de prueba / caso de prueba
8. Roles (o grupos que van a realizar partes de la funcionalidad)
9. 'Optimizar ahora' vs 'optimizar más tarde'
10. Criterios de aceptación identificados
11. Elementos en función de los requisitos de usabilidad y experiencia usuario



Ejemplo de patrones de División de Historias de Usuario

US: "Como cliente frecuente, quiero elegir y pagar los productos en mi carrito de compra para recibir mis productos en casa."

Desglosado por pasos del Flujo de trabajo

- Como cliente existente puedo iniciar sesión con mi cuenta.
- Como cliente puedo revisar y confirmar mi pedido.
- Como cliente, puedo pagar mi pedido con una transferencia bancaria.
- Como cliente frecuente, puedo pagar mi pedido con tarjeta de crédito.
- Como cliente frecuente recibo un correo electrónico de confirmación con mi pedido y fechas de entrega.

Desglosado por tipos de datos o parámetros

- Como cliente frecuente, puedo buscar un producto entre mis compras anteriores, por lo que puedo encontrar rápidamente.
- Como cliente, puedo buscar productos en un rango de precios, para que los resultados de búsqueda sean más relevantes.
- Como cliente, puedo buscar productos en un grupo de productos, de modo que los resultados de búsqueda sean más relevantes.

Desglosado por reglas de negocio, ya sean comerciales implícitas o explícitas

- Como propietario de una tienda, puedo rechazar pedidos de menos de 10 dólares, porque no son rentables.
- Como propietario de la tienda, puedo rechazar clientes fuera X provincias porque los gastos de envío hacen que estos pedidos no son rentables.
- Como propietario de la tienda, puedo reservar productos en stock por 48 horas, para que otros clientes vean el stock realista.
- Como propietario de la tienda, puedo cancelar automáticamente los pedidos por los que no he recibido el pago dentro de las 48 horas, incrementar mi stock y puedo venderlos a otros clientes

Desglose los elementos en función de los requisitos de usabilidad

- Rendimiento : Velocidad de ejecución de los pedidos y pagos
- Facilidad de Aprendizaje : Tiempo requerido por el usuario para hacer compras frecuentes
- Flexibilidad : Acomodarse a cambios de interfaz
- Actitud : Generar actitud positiva en los usuarios, completando encuestas de la experiencia por mail

Criterios de Aceptación

- Los criterios de aceptación **son las condiciones que un producto de software debe satisfacer para ser aceptado por un usuario o cliente**. Se los define como un conjunto de sentencias redactadas de tal manera que conduzcan a una respuesta clara de aceptado/rechazado.
- Formatos para escribir criterios de aceptación:
 - **Comprobar** [Criterios]
 - **Demostrar** [Comportamiento esperado]
 - **Verificar que cuando** [Rol] **hace** [Acción] **consigue** [Resultado / Comportamiento esperado]
 - **Dado que** [Contexto/escenario] **y adicionalmente** [Contexto] **cuando** [Evento/Comportamiento] **entonces** [Resultado / Comportamiento esperado]

Criterios de Aceptación



- La **sintaxis de Gherkin** es la siguiente:
 - **(Scenario) Escenario** [Número de escenario] [Titulo del escenario],
 - **(Given) Dado que** [Contexto] y adicionalmente [Contexto],
 - **(When) cuando** [Evento],
 - **(Then) entonces** [Resultado / Comportamiento esperado]
- Derivado de esta sintaxis los elementos de los criterios de aceptación son:
 - **Número de escenario:** Número (ejemplo 1, 2, 3 ó 4), que identifica al escenario asociado a la US.
 - **Título del escenario:** Describe el contexto del escenario que define un comportamiento.
 - **Contexto:** Proporciona mayor descripción sobre las condiciones que desencadenan el escenario.
 - **Evento:** Representa la acción que el Usuario ejecuta, en el contexto definido para el escenario.
 - **Resultado / Comportamiento esperado:** Dado el contexto y la acción ejecutada por el Usuario, la consecuencia es el comportamiento del sistema en esa situación.

Qué es Gherkin?

OW

Criterios de Aceptación: SMART

- Para medir la calidad de un criterio de aceptación se utiliza el método **SMART** en el que se han de cumplir en lo máximo posible los siguientes criterios:
 - S** - Specific (Específicos)
 - M** - Measurable (Medibles)
 - A** - Achievable (Alcanzables)
 - R** - Relevant (Relevantes)
 - T** - Time-boxed (Limitados en el tiempo)

Los criterios de aceptación son un conjunto de sentencias, cada una con un resultado de pase / error claro, que se pueden medir y especificar requisitos funcionales y no funcionales.



Ejemplo

Partimos de la **siguiente US**

- **Como** Cliente
- **Quiero** retirar dinero del cajero automático
- **Para poder** evitar ir al banco a hacer una cola

y escribimos los **criterios de aceptación en Gherkin**

Escenario 1: Cuenta tiene crédito

Dado que la cuenta tiene crédito

y que la tarjeta es válida

y que el cajero tiene dinero disponible

Cuando el cliente pide dinero

Entonces la cuenta es debitada

y el dinero es entregado al cliente

y el cliente recupera su tarjeta

Escenario 2: La cuenta excede el límite negativo acordado con el banco

Dado que la cuenta excede el límite negativo acordado con el banco

y que la tarjeta es válida

Cuando el cliente pide dinero

Entonces el cajero muestra un mensaje negando el pedido

y el dinero no es entregado al cliente

y el cliente recupera su tarjeta



Ejemplo

Historia Negociable 1

Como: Bloguero

Quiero: hacer una entrada al blog

Para: posicionarme como experto en un tema específico

Criterios de Aceptación:

- Debo ser capaz de publicar contenido multimedia (imágenes y video)
- El texto de la entrada debe ser enriquecido (que permita enlaces Web, formato, etc)
- La entrada se debe poder compartir vía redes sociales
- La entrada se debe poder imprimir
- La entrada se debe poder enviar vía correo electrónico

ejemplo

US: como usuario, quiero poder registrarme en el servicio para poder comenzar a comprar en línea.

Criterios de aceptación:

- Los usuarios solo pueden enviar un formulario rellenando todos los campos obligatorios.
- El Correo electrónico que proporcione el Usuario no debe ser proporcionado por un servicio de correo electrónico gratuito.
- Las presentaciones de la misma IP solo se pueden hacer tres veces en un plazo de 30 minutos.
- Los usuarios reciben correos electrónicos de notificación después de registrarse correctamente.,

ejemplo

US: como usuario, puedo acceder a una notificación en mi dispositivo inmediatamente después de recibirla.

Criterios de aceptación:

- deslizar / tocar una notificación lleva al usuario directamente al mensaje.
- La Vista muestra la conversación — si el nuevo mensaje era una respuesta, se muestra encima del original.
- El recuento de mensajes se actualiza.
- Un mensaje se marca como leído después de que se muestra.

Como [cliente habitual], **quiero** [ver productos relacionados] **para** [ver si hay otros productos que me puedan interesar]

Condiciones de completitud

- Los productos estarán ordenados por valoración y margen de beneficio.
- Cuando el usuario haga clic en un producto, se desplegará el detalle.
- Etc.

Prioridad

70

Coste

5

http://tamr1.atac.flickr.com/55147874576_8a45307983.jpg

www.agile-spain.com

viernes 18 de enero de 13

Criterios de Aceptación

Ejemplo

"Como usuario se me debe requerir una validación antes de utilizar el sitio"

Criterios de Aceptación:

- El usuario esta logueado solo cuando se proporcionen credenciales apropiadas
- Esta disponible una opción "recordarme".
- El usuario puede requerir un recordatorio de contraseña.
- El usuario es bloqueado luego de 3 intentos fallidos

"Como comprador del Sitio Web quiero poder pagar con una tarjeta de crédito para poder confirmar inmediatamente mi compra"

Criterios de Aceptación:

- Acepta Visa, Diners, MasterCard
- Validar Nro de CC cuando sea ingresado
- Validar fecha de expiración y CVV
- Validar la dirección de facturación
- Generar mensajes de satisfacción y fallo luego del procesamiento.

"Como contador quiero que los reportes automatizados se ejecuten al final del mes para que los reportes estén listos al llegar a la oficina"

Criterios de Aceptación:

- Si hay un error con la generación del reporte, el Sistema necesita notificar a soporte de producción con un ticket.
- El reporte necesita ser generado como PDF y auto-impreso.
- La selección de auto-impresión necesita ser configurable por reporte
- El Sistema debería enviar el reporte solo a la impresora configurada.
- Si la impresora tiene un error (falta papel, trabado, etc.) el usuario debería arreglarlo.

Criterios de Aceptación: puntos clave

- Los criterios de aceptación **son las condiciones que un producto de software debe satisfacer para ser aceptado por un usuario, cliente o, en el caso de la funcionalidad a nivel de sistema, el sistema consumidor.**
- Los criterios de aceptación **deben documentarse y completarse antes del inicio de un proyecto**, ya que el equipo y el cliente deben acordar qué resultados cumplirán con los requisitos del cliente.
- recuerde que los criterios de aceptación **deben ser una expresión de intención, no una decisión final.**
- Los criterios de aceptación efectiva definen una **cantidad mínima razonable de funcionalidad.**
- Los buenos criterios de aceptación deben **describir el alcance del trabajo** para que los desarrolladores puedan planificar y estimar adecuadamente sus esfuerzos.
- Los criterios de aceptación deben estar escritos en lenguaje sencillo.
- Asegúrese de **comunicar** sus criterios de aceptación a las partes interesadas y los miembros del equipo y llegar a un acuerdo mutuo.
- Al escribir los criterios de aceptación, **evite usar» no», conjunciones y absolutos inalcanzables.** Formular oraciones usando la voz activa.,

Criterios de Aceptación: reglas de escritura

REGLAS	Descripcion	User Story Ejemplo	Criterio de aceptación
Evite «no»	» no «significa» en ningún caso » y, por lo tanto, no será suficiente tiempo para verificar el cumplimiento de tal condición., Si reescribe el requisito sin usar «no», será más claro y, lo más importante, verificable.	no: como usuario, no quiero tener que introducir mi contraseña cada vez que acceda a mi cuenta. hacer: como usuario, quiero que mi contraseña sea recordada y completada automáticamente para que pueda acceder a mi cuenta sin volver a ingresar mi contraseña.	no: El sistema no debe fallar. Do: el sistema debe tener una disponibilidad de no menos del 90%,
Uso de la voz activa	es cuando el sujeto de una oración es el ejecutante de la acción. Si la entidad responsable de ejecutar la acción no está claramente indicada, no estará claro quién o qué debe realizar la acción, y será más difícil para Usted verificar si se cumple un requisito.,	no: como comprador en línea, Quiero que se apliquen filtros para que pueda encontrar lo que quiero. Do: como usuario, quiero aplicar filtros de búsqueda para que pueda encontrar elementos.	no: se debe confirmar la identidad del cliente. (No está claro quién o qué es responsable de confirmar la identidad del cliente.) Do: El Accounting_System debe confirmar la Customer_Identity. (Tenga en cuenta que las definiciones de los Términos «Accounting_System» y «Customer_Identity» deben agregarse al glosario.,)
Evite usar pronombres (especialmente los indefinidos)	Use sustantivos en lugar de pronombres cuando se refiera a elementos a los que se hace referencia en otros requisitos. Los pronombres deben evitarse porque pueden introducir ambigüedad., Esto es especialmente importante cuando los criterios de aceptación se almacenan en Herramientas de gestión de requisitos (como Jira) como declaraciones separadas que no están necesariamente organizadas. Siempre use sustantivos en lugar de pronombres y evitará este problema.	no: como miembro del sitio, quiero compartir información sobre mí para que otros puedan verla. Do: como miembro del sitio, quiero agregar una descripción de perfil para que otros puedan aprender sobre mí.	no: el controlador debe enviar al conductor el itinerario del día., Debe entregarse al menos 8 horas antes del turno. Do: El controlador debe enviar el Driver_Itinerary del día al conductor al menos 8 horas antes del Driver_Shift.

REGLAS	Descripcion	User Story Ejemplo	Criterio de aceptación
Evitar las conjunciones	las Conjunciones son palabras y frases tales como «y», «o», «pero», y «así como» que combinan las oraciones simples a los más complejos., Su uso en los requisitos suele ser una señal de que un requisito se puede dividir en varios requisitos separados. Si se puede usar: «And, «»or,» Y » not » se pueden usar para describir condiciones lógicas y agregar calificadores.	no: como diseñador de interfaz de usuario, quiero crear y ver un problema para saber qué probar. Do: como diseñador de UI, quiero crear un problema para saber qué probar. / Como diseñador de UI, quiero ver un problema para saber qué probar.	no: el Usuario debe ser de confianza o no. Do: El Security_System debe categorizar a cada usuario como de confianza o Not_Trusted.,
Evite los absolutos inalcanzables	un absoluto (como la disponibilidad del 100%) es inalcanzable. Piense en cómo verificar el indicador: ¿será posible demostrar que el nivel de disponibilidad del sistema es exactamente del 100%?, E incluso si se pudiera crear un sistema de este tipo, ¿puede permitírsele? evite las palabras «todo», «siempre» y «nunca», ya que verificar tales requisitos absolutos requerirá un número infinito de pruebas.	no: como viajero, quiero saber mi ubicación precisa actualizada en tiempo real para no perderme. («Tiempo Real» se puede interpretar de diferentes maneras. Por ejemplo, puede verse como un absoluto (la ausencia de cualquier retraso), que no se puede lograr y que no es verificable.,) Do: como viajero, quiero saber mi ubicación precisa, actualizada cada segundo, para que no me pierda.	no: el sistema debe tener 100% de disponibilidad. (100% es un absoluto que no se puede alcanzar y no se puede verificar.) Do: el sistema debe tener una disponibilidad de al menos 98%.

Como Neil Armstrong
Quiero Un traje espacial
Para dar un paseo por la
superficie lunar

PRIORIDAD 10

TAMAÑO 15

CRITERIO DE ACEPTACIÓN

Dado un astronauta que se
encuentra en un viaje a la luna
cuando aluniza y desciende
de la nave espacial
entonces no se congela y
puede respirar.

COMO barman designado de la fiesta

QUIERO mezclar perfectamente los ingredientes del Fernet con Cola

PARA que no se desperdicien bebidas y salgan buenos tragos

DADO que las Cocas las van a ir trayendo los invitados,

CUANDO lleguen cada invitado

ENTONCES las pondré en la heladera y el freezer según lo caliente que estén

DADO

CUANDO

ENTONCES

DADO

CUANDO

ENTONCES

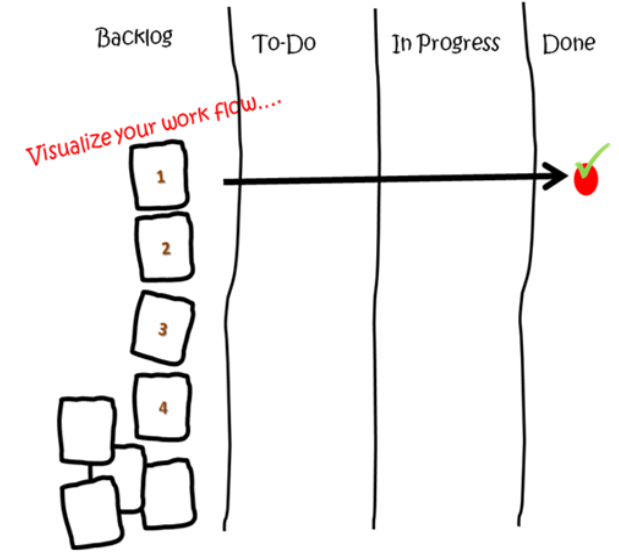
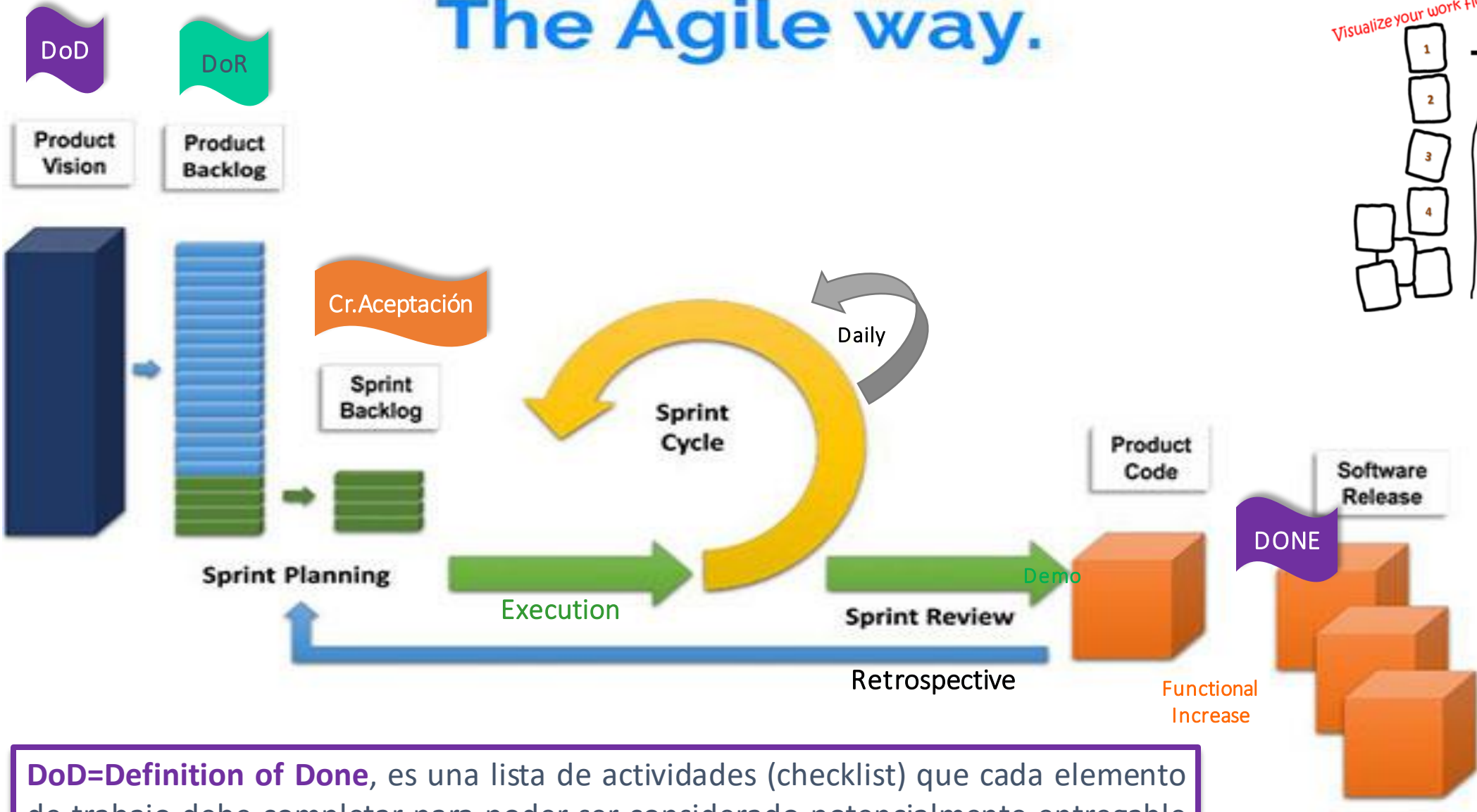
DADO

CUANDO

ENTONCES



The Agile way.



DoD=Definition of Done, es una lista de actividades (checklist) que cada elemento de trabajo debe completar para poder ser considerado potencialmente entregable para un cliente dado y conforma un "entendimiento compartido de lo que significa que el trabajo está completado"

Criterios de aceptación

DoR ó Definición de Ready

DONE o DoD

La definición de "**Terminado o DONE**" es un entendimiento compartido de **completado para el producto**.

- Debe ser **visible y universalmente entendido y acordado** para fomentar la transparencia.
- **Es el denominador común de la calidad del producto.**
 - Por ejemplo, un equipo de desarrollo que entrega una funcionalidad al Product Owner cree que ya ha terminado, el Product Owner cree que eso significa que junto con esa funcionalidad se ha elaborado la documentación asociada a la funcionalidad. El equipo de desarrollo no ha entregado esa documentación y no entiende porque se requería. Es un clásico problema de comunicación que podría ser resuelto por la definición de terminado.
- En Scrum, el encargado de elaborar los elementos del Product Backlog es el **Product Owner**, aunque Scrum permite que este se apoye en integrantes del equipo de desarrollo que puedan colaborar con él.
- El **Scrum Master** tiene la tarea de ayudar al equipo Scrum (Product Owner y desarrolladores) a aplicar técnicas para que los elementos del Product Backlog sean claros y concisos.



Qué debería incluir la definición de terminado?

- Estándares de la organización de desarrollo, si estos estándares no existen entonces el equipo de desarrollo puede elaborarlos.
- Cubrir todas las actividades del proceso de desarrollo, por ejemplo actividades de análisis, de diseño, de codificación, de pruebas y despliegue entre otros.
- Deberían servir para que el equipo de desarrollo considere el esfuerzo de implementarlos durante la estimación del Product Backlog.

Cuales son los requisitos mas comunes definidos en el Definition of Done?

- La tarea debe cumplir con los estándares de calidad definidos.
- La tarea debe haber sido validada y verificada.
- La tarea debe haber sido puesta en producción o haberse hecho la petición de pase a producción.



Qué debería incluir la definición de terminado?

Ejemplos

- Todas las pruebas unitarias y funcionales son correctas.
- Todos los criterios de aceptación se cumplen.
- OK del equipo: UX, desarrollador, Product Owner, etc.
- Pruebas en dispositivos/navegadores pasada.
- Se han corregido todos los bugs.
- Entorno preparado para la subida a producción.
- Calidad de código (Por ejemplo deuda técnica medida con SonarQube o usar NDepends)
- Integración y despliegue continuo, sin errores
- Refactoring terminado y probado
- Pruebas de aceptación del usuario, validados y aceptados por el PO
- Uso TDD, unit test ejecutado sin errores
- Pruebas de integración finalizadas sin errores
- Internacionalización a varias culturas
- Documentación requerida por la organización
- Pruebas de regresión finalizadas sin errores
- Pruebas de rendimiento y escalabilidad pasadas
- Pruebas de seguridad sin errores
- Capacitación en lecciones aprendidas y buenas prácticas de programación y testing

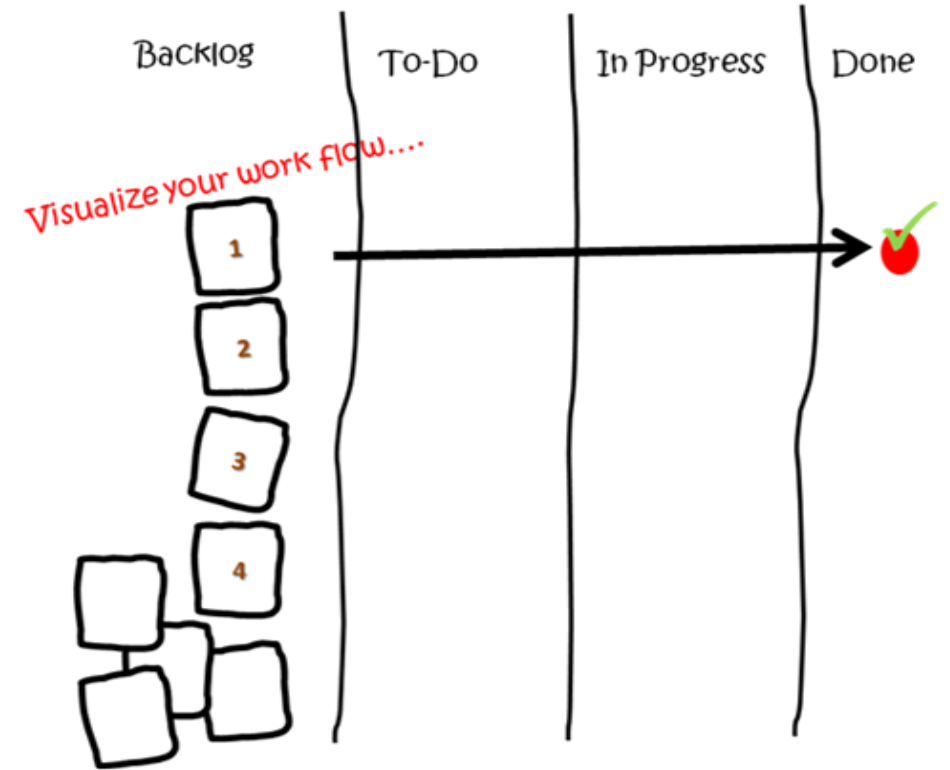
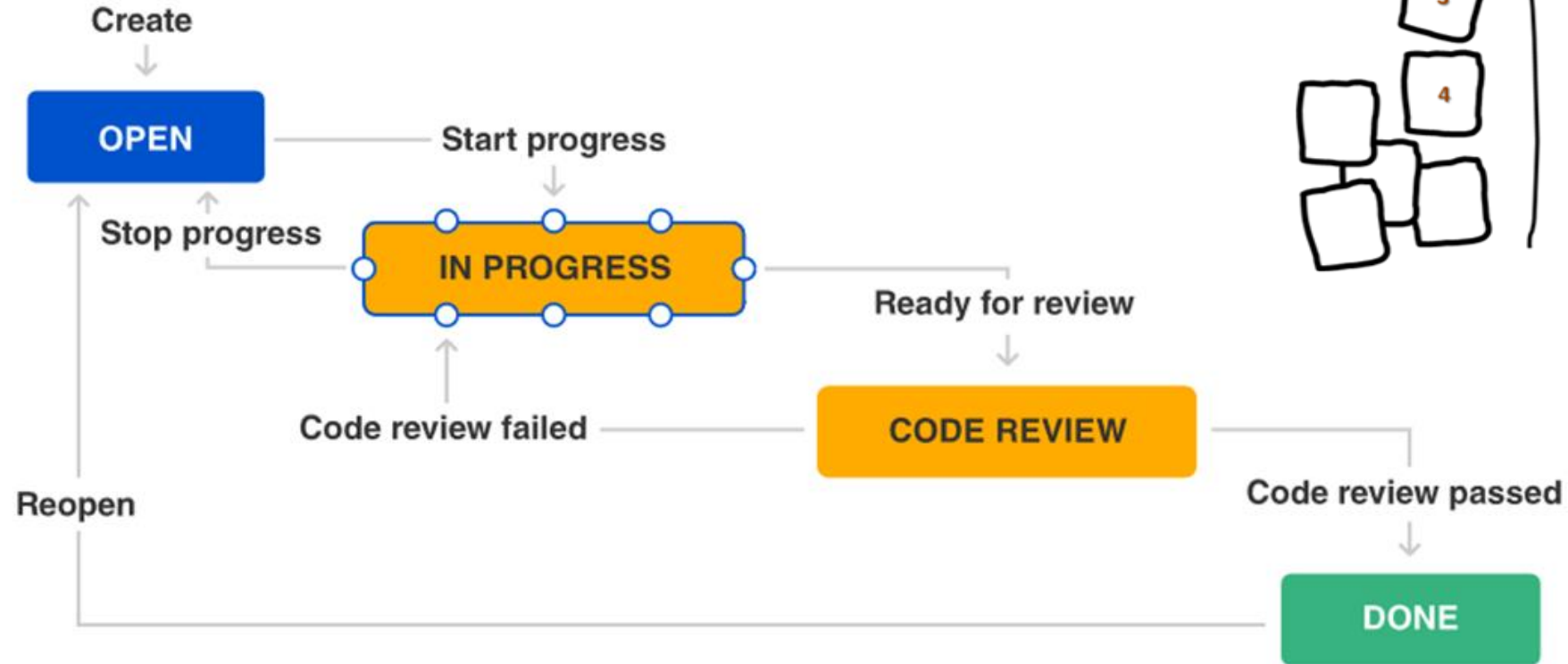


Preguntas que pueden ayudarnos a mejorar la definición de Terminado (Done)

- Existe una definición de Done? Hay un check list a seguir?
- Ha sido acordada con la organización de desarrollo?
- Ha sido acordada con el Product Owner?
- Sigue estándares de calidad de desarrollo de software adecuados?
- Cubre aspectos para todas las actividades que permiten producir un incremento usable de producto?
- Es visible y está disponible para el equipo?
- Está siendo usada durante el transcurso del Sprint?
- Ha sido revisada y actualizada durante el Sprint Retrospective?
- Los criterios de Done incluyen:
 - los estándares de desarrollo de la organización?
 - las prácticas de DevOps?
 - elementos requeridos por la organización como por ejemplo documentación?



Definición de terminado o DONE o DoD



Refinamiento: repaso

El refinamiento es una reorganización del Product Backlog, es decir, una actualización del mismo.

El refinamiento nos ayuda a dejar a “ready” los ítems y nos permite tener esas conversaciones que necesitamos para no retrasar al equipo o todo aquello que afecte a la entrega de valor.

Por tanto, tenemos que buscar el momento en el que refinar y trabajar que los ítems estén listos.

- La ventaja del refinamiento Scrum es que **no prescribe** cómo hay que hacerlo y permite espacio para decidir.
- Hay equipos que redistribuyen los ítems para refinar cada uno con el Product Owner por separado. Después, cada uno debe explicárselo a sus compañeros.



Refinamiento

Refinamiento

Recortar

Cada elemento debe estar lo suficientemente ajustado para poder entrar en el Sprint.

La hoja para dividir historias ayuda a los equipos a explorar las diferentes posibilidades de fraccionar cada elemento.

Estimar

Aun cuando un elemento no está del todo claro, se puede estimar. La forma como se haga dependerá de muchos factores, pero es mejor hacerlo conforme sea la situación.

Entre las técnicas de estimación más usadas tenemos:

Redactar criterios de aceptación

El solo uso de la plantilla de historia de usuario puede ser suficiente para que el equipo de desarrollo tenga un elemento listo.

El nivel de detalle al escribir los criterios de aceptación depende del elemento en mano y del nivel de conocimiento y experiencia de las personas involucradas.



1. Talla de camiseta

Todo el mundo entiende el concepto de pequeño, mediano o grande en cuanto a tallas de camisetas, por lo que se puede trasladar esto a estimaciones de refinamiento de forma sencilla y rápida. Esta es una excelente técnica para estimar rápidamente casi cualquier elemento.



2. Estimación mágica

Se busca estimar varios elementos de manera rápida. Utilizando esta técnica, se puede tener una idea rápida del esfuerzo percibido en los elementos del Product Backlog.

Primero se reúne un equipo Scrum y el Product Owner imprime entrega cada US o PBI (ítem de Product Backlog separadas. El equipo es quien debe hacer la estimación sin comunicación entre ellos en al menos 15 minutos.



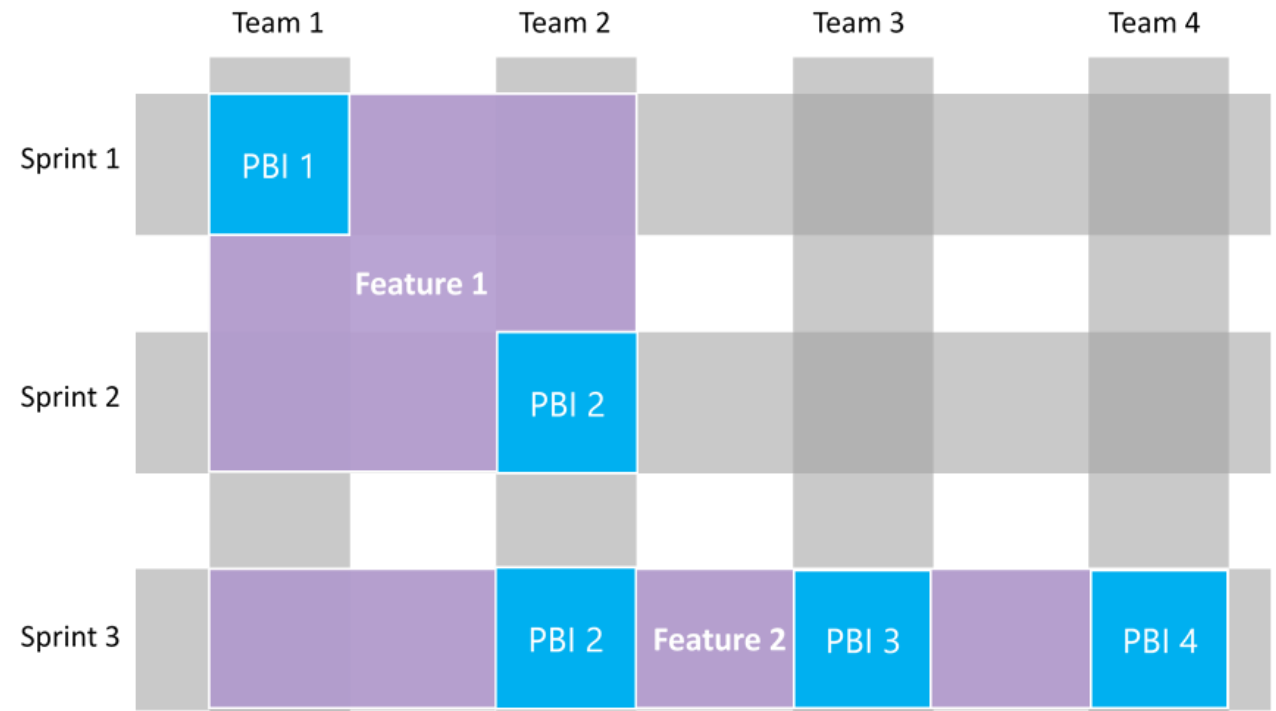
3. Planning Póquer

Esta técnica tiene su origen en el método Wideband Delphi, por lo que es una variación del mismo.

Requiere de mucho tiempo pero es muy eficaz

Refinamiento del Backlog o Grooming

- Es una reunión que no es una ceremonia de scrum original, **es una actividad propuesta por la guía de Scrum útil para refinar la lista de PBIs o Product Backlog ítem y que consiste en trabajar sobre las hipótesis de requerimientos, definir las, analizarlas, añadirles detalles, granularizarlas, estimarlas y priorizarlas.**
- Es un proceso continuo que hace el Product Owner, pero formalmente como reunión se desarrolla con participación del Equipo en colaboración para examinar y revisar los elementos del Product Backlog.
- En esta reunión la responsabilidad recae en el PO y el Equipo colabora.



Feature=Función